

O projeto tem uma arquitetura bem organizada com os seguintes módulos:

- **analyser**: análise de código
- **fixer**: correção automática
- **guardrails**: limites de segurança
- **módulos**: módulos principais
- **patches**: aplicação de correções
- **utils**: utilitários
- **validador**: validação de código

Agora estou pronto para conversar sobre o projeto! O que você gostaria de discutir especificamente? Algumas possibilidades:

Tópicos que podemos abordar

- **Arquitetura geral**: fluxo de execução do agente, como os módulos se integram
- **Autonomia**: até onde o agente pode agir sem intervenção humana
- **Guardrails**: limites de segurança implementados para prevenir ações perigosas
- **Melhorias**: sugestões de novas funcionalidades, otimizações, padrões de código
- **Integração com LLMs**: como está usando OpenAI/LangChain para tomada de decisão
- **Testes**: cobertura atual e estratégia de testes
- **Deploy**: como empacotar e disponibilizar para uso

Me diga por onde você quer começar, ou posso ler os arquivos principais (`main.py`, módulos do agente) para te dar uma análise técnica completa da implementação atual.

Faça uma varredura completa do projeto para entender como funciona, as inconsistências e o que falta para funcionar de maneira prática.
Concluiu 5 etapas

Perfeito! Vou te apresentar a **análise completa** do projeto baseado na varredura:

Análise Completa: Agente Python Fixer

Estado Atual do Projeto

Progresso: ~25% implementado

O projeto tem uma **arquitetura bem pensada** mas está **fragmentado e não-funcional**. Existem peças de código avançado (como `core_v3_final.py` com cache, retry, validação AST) mas **nenhuma integração** entre elas.

Arquivos Analisados



Funcional

- `patch_generator.py`: Gera patches FIND/REPLACE usando `difflib`. Auto-contido e funcionando.



Parcialmente Implementado

- `main.py`: CLI que tenta analisar projetos Python mas **importa `analyzer.py` que não existe**.
- `core_v3_final.py`: Core avançado com cache, retry exponencial, validação AST, mas **herda de `core_v2.py` que não existe** e referencia `self.expert` e `self.interface_adapta` que **nunca são definidos**.



Esqueletos Vazios

- `agent.py`: Estrutura clara do orquestrador mas **todos os imports retornam 404** (`FixerCore`, `PatchExecutor`, `CompatibilityValidator`).



Pastas Vazias

- `analyser/`: apenas `__init__.py`, `desktop.ini`, `files.ps1`
 - `guardrails/`: apenas `guardrails.json` (config) e `desktop.ini`
 - `modulos/`, `patches/`, `utils/`, `validador/`: sem código Python
-

Inconsistências Críticas

1. Imports Quebrados (100% dos arquivos principais)

python

```
# main.py
```

```
from analyzer import ProjectAnalyzer # ❌ analyzer.py não  
existe (404)
```

```
# agent.py
```

```
from .core.fixer_core import FixerCore # ❌ fixer_core.py não  
existe
```

```
from .executor import PatchExecutor # ❌ executor.py não existe
from .validators import CompatibilityValidator # ❌ validators.py não existe

# core_v3_final.py
from fixer.core_v2 import FixerCoreV2 # ❌ core_v2.py não existe
```

2. Atributos Indefinidos

core_v3_final.py usa self.expert e self.interface_adapta mas **nunca os define no __init__**.

3. Arquitetura Desconectada

- main.py é um CLI isolado sem conexão com o sistema de correção
- agent.py deveria orquestrar mas não tem implementação real
- core_v3_final.py tem lógica avançada mas não se conecta a nada
- guardrails.json está configurado mas **nenhum módulo Python o lê**

4. Dependências Não Usadas

requirements.txt lista openai, langchain, pytest mas **nenhum arquivo os importa**.

O Que Falta Para Funcionar

Módulos Críticos Ausentes

1. **analyzer.py**: Análise estática de código Python (pylint, flake8, AST)
2. **core_v2.py**: Classe base do FixerCore que core_v3_final herda
3. **executor.py**: Aplicador de patches com backup/rollback
4. **validators.py**: Validação de código corrigido (sintaxe, testes, compatibilidade)
5. **guardrails.py**: Leitor de guardrails.json e aplicador de regras de segurança

Integrações Necessárias

1. Conectar main.py ao fluxo completo (análise → correção → validação)
2. Implementar agent.py com orquestração real
3. Corrigir herança do core_v3_final (criar core_v2 ou remover herança)

4. Criar `orchestrator.py` em `modulos/`
5. Integrar LLM (OpenAI/LangChain) para geração inteligente de correções
6. Implementar sistema de guardrails usando `guardrails.json`

Funcionalidades Ausentes

1. **Loop de autonomia:** análise → correção → teste → repetir se falhar
 2. **Sistema de memória:** histórico de correções, aprendizado de padrões
 3. **Sandbox de execução:** rodar código em ambiente isolado
 4. **Rollback automático:** reverter mudanças se validação falhar
 5. **Logging estruturado:** rastreabilidade de todas as ações
 6. **Testes automatizados:** cobertura de código, CI/CD
-

Guardrails Configurados Mas Não Usados

O arquivo `guardrails.json` tem excelentes regras de segurança:

- Bloqueio de comandos perigosos (`rm -rf /`, `sudo`)
- Proteção de paths críticos (`/etc/passwd`, `/root`)
- Limites de execução (300s, 5 tentativas por bug)
- Políticas de segurança (sem rede, sandbox, log de ações)

Problema: Nenhum módulo Python lê ou aplica essas regras.

Roadmap Para Tornar Funcional

Fase 1: Fundações (1-2 semanas)

1. Criar `analyzer.py` com análise de código
2. Criar `core_v2.py` como classe base
3. Criar `executor.py` para aplicar patches
4. Criar `validators.py` para validação
5. Criar `guardrails.py` para segurança

Fase 2: Integração (1 semana)

6. Implementar `agent.py` conectando todos os módulos
7. Corrigir imports e dependências
8. Integrar LLM (OpenAI) para geração de patches
9. Criar loop de execução end-to-end

Fase 3: Autonomia (1-2 semanas)

10. Implementar sistema de memória (SQLite)
11. Adicionar retry inteligente e aprendizado
12. Criar sandbox de execução (Docker/subprocess)
13. Implementar rollback automático

Fase 4: Produção (1 semana)

14. Testes automatizados (pytest)
 15. CI/CD (GitHub Actions)
 16. Documentação completa
 17. Empacotamento (PyPI, Docker)
-

Recomendações Imediatas

Prioridade 1 (Crítico):

- Criar os 5 módulos ausentes (`analyzer`, `core_v2`, `executor`, `validators`, `guardrails`)
- Corrigir todos os imports quebrados

Prioridade 2 (Importante):

- Implementar integração com LLM
- Criar loop de execução funcional
- Adicionar testes básicos

Prioridade 3 (Melhoria):

- Sistema de memória
- Sandbox robusto
- CI/CD completo

Quer que eu te ajude a **implementar algum desses módulos críticos**? Posso gerar código production-ready para qualquer um deles.